

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Operating System Lab (CO2018)

Class: CC02 - Group: MyOS

LAB REPORT

SIMPLE OPERATING SYSTEM

Instructor:	Hoàng Lê Hải Thanh	
Students:	Dương Minh Nhật	2452889
	Võ Minh Trí	2453323
	Nguyễn Thái Hoài Nam	2452794
	Nguyễn Trọng Nhân	2452883
	Lê Tuấn Kiệt	2452632

Ho Chi Minh City, May, 2026



MEMBERLIST

No.	Name	ID	Task	Performance
1	Dương Minh Nhật	2452889	Memory Management, System Calls	100%
2	Võ Minh Trí	2453323	Memory Management	100%
3	Nguyễn Thái Hoài Nam	2452794	CPU Execution, Scheduling	100%
4	Nguyễn Trọng Nhân	2452883	Multi-level Paging, OS Init	100%
5	Lê Tuấn Kiệt	2452632	Scheduling, OS Threading	100%

Index

1	OS Initialization & Threading Architecture	3
1.1	Configuration Parsing	3
1.2	Bootstrapping the Kernel	4
1.3	Multi-Threading Simulator	4
1.4	The Program Loader (loader.c)	5
2	The CPU Execution Cycle	5
2.1	Process Dispatching	5
2.2	Instruction Execution (run())	7
3	Scheduling	9
3.1	Implementation Breakdown	9
3.1.1	Ready Queue Management (queue.c)	10
3.1.2	Scheduling Logic (sched.c)	10
3.2	Code Walkthrough: get_mlq_proc()	11
3.3	Kernel Structure Pointer Setup	13
3.4	Thread Safety	13
3.5	Gantt Diagram	13
4	Memory Management	14
4.1	Virtual Memory Architecture	14
4.2	The ALLOC Operation (liballoc)	15
4.3	The FREE Operation (__free)	16
4.4	READ/WRITE & Page Replacement Strategy	17
4.5	Detailed Page Fault Mechanics	18
4.6	Kernel Memory Operations	19
4.6.1	KMALLOC — Physically Contiguous Allocation	19
4.6.2	User/Kernel Address Tagging	20
4.6.3	Slab Cache: KMEM_CACHE_CREATE & KMEM_CACHE_ALLOC	20
4.6.4	COPY_FROM_USER & COPY_TO_USER	21
5	System Call Integration	21
5.1	Register-Based Argument Passing	22
5.2	System Call Dispatching and Protection	22
5.3	PID-Based Kernel Traversal	23
5.4	Wrapper Library (libstd.c)	24

6	Multi-level Paging (64-bit Scheme)	25
6.1	Address Translation Process	25
6.2	Canonical Address Handling	26
6.3	PTE Bit Layout	26
6.4	Hierarchical Entry Population	27
6.5	The vmmap_pgdmemset System Call	28
6.6	Quantitative Statistics (Required by Specification)	29
7	Questionnaire & Practical Examples	30
8	Extended Execution Trace Analysis	40
8.1	Execution Trace Example (Page Table Metadata Inspection)	40
8.2	Privilege Isolation & Concurrency Trace Analysis	41

1 OS Initialization & Threading Architecture

Before any scheduling or memory management can occur, the operating system must bootstrap itself. The `os.c` file serves as the entry point (`main()`) for our simulated environment. It simulates a symmetric multiprocessing (SMP) architecture using POSIX threads (`pthread`).

1.1 Configuration Parsing

The OS requires an input configuration file (e.g., `os_1.conf`) to define the hardware limitations and process queue.

```
1 static void read_config(const char * path) {
2     FILE * file = fopen(path, "r");
3     fscanf(file, "%d %d %d\n", &time_slot, &num_cpus, &num_processes);
4
5     /* Deterministic mode for test replay */
6     if (getenv("OSSIM_DETERMINISTIC") != NULL && strcmp(getenv("
OSSIM_DETERMINISTIC"), "1") == 0)
7         num_cpus = 1;
8
9     /* Legacy vs Paging Memory Parsing */
10    memramsz = 268435456UL; // Default 256MB
11    memswpsz[0] = 16777216UL; // Default 16MB Swap
12    // ... logic to read memory boundaries and process list ...
13 }
```

Listing 1: Configuration parsing in `os.c`

The OS uses the `OSSIM_DETERMINISTIC` environment variable to force the system into a single-CPU mode. This is crucial for debugging race conditions. When testing scheduling policies, thread dispatch interleaving differences across multiple CPUs can cause outputs to be non-deterministic. By forcing `num_cpus = 1`, we guarantee repeatable tests.

1.2 Bootstrapping the Kernel

The main function initializes the kernel memory devices before spawning threads.

```
1 #ifdef MM_PAGING
2     struct memphy_struct mram;
3     struct memphy_struct mswp[PAGING_MAX_MMSWP];
4
5     init_memphy(&mram, memramsz, rdmflag);
6     for(sit = 0; sit < PAGING_MAX_MMSWP; sit++)
7         init_memphy(&mswp[sit], memswpsz[sit], rdmflag);
8
9     struct mmpaging_ld_args *mm_ld_args = malloc(sizeof(struct
10 mmpaging_ld_args));
11     mm_ld_args->mram = &mram;
12     mm_ld_args->mswp = &mswp;
13 #endif
14 init_scheduler();
```

Listing 2: Kernel memory initialization

We instantiate a single physical RAM (`mram`) and an array of Swap devices (`mswp`). These are explicitly defined in user-space but passed into the kernel structures later.

1.3 Multi-Threading Simulator

The system spawns N CPU threads and 1 Loader thread.

```
1 pthread_create(&ld, NULL, ld_routine, (void*)mm_ld_args);
2
3 for (i = 0; i < num_cpus; i++) {
4     pthread_create(&cpu[i], NULL, cpu_routine, (void*)&args[i]);
5 }
6
7 for (i = 0; i < num_cpus; i++) {
8     pthread_join(cpu[i], NULL);
9 }
10 pthread_join(ld, NULL);
```

Listing 3: Spawning OS Threads

Loader Routine (ld_routine)

The loader acts as the "Disk to Memory" mechanism. It wakes up at specific time intervals (checked via `current_time()`), reads the program text files, creates new `pcb_t` blocks, initializes their page directories, and pushes them into the scheduler's ready queue via `add_proc()`.

1.4 The Program Loader (loader.c)

To execute files, the OS must parse text-based instruction files into runnable `pcb_t` structures. The `load()` function reads the process path and translates human-readable opcodes into internal enum `ins_opcode_t`.

```
1 static enum ins_opcode_t get_opcode(char * opt) {  
2     if (!strcmp(opt, "calc")) return CALC;  
3     else if (!strcmp(opt, "alloc")) return ALLOC;  
4     else if (!strcmp(opt, "free")) return FREE;  
5     else if (!strcmp(opt, "read")) return READ;  
6     else if (!strcmp(opt, "write")) return WRITE;  
7     // ...  
8 }
```

Listing 4: Opcode Parsing

The loader then parses the required arguments (registers, offsets, sizes) depending on the opcode type. For instance, `READ` requires 3 arguments (`arg_0`, `arg_1`, `arg_2`), whereas `FREE` only requires 1. This is stored directly in the `proc->code->text` array for the CPU to fetch.

2 The CPU Execution Cycle

Once the processes are loaded into the ready queue, the CPU threads execute the `cpu_routine`. This routine acts as a perpetual Fetch-Decode-Execute cycle.

2.1 Process Dispatching

The CPU constantly polls the scheduler for work.

```
1 while (1) {  
2     if (proc == NULL) {
```

```
3     proc = get_proc(); // Fetches highest priority process
4     if (proc == NULL) {
5         next_slot(timer_id);
6         continue;
7     }
8     } else if (proc->pc == proc->code->size) {
9         // Process finished executing all instructions
10        printf("\tCPU %d: Processed %2d has finished\n", id, proc->pid)
11        ;
12        remove_running_proc(proc);
13        free(proc);
14        proc = get_proc();
15        time_left = 0;
16    } else if (time_left == 0) {
17        // Time slice exhausted - Preemption
18        printf("\tCPU %d: Put process %2d to run queue\n", id, proc->
19        pid);
20        put_proc(proc); // Put back to its priority queue
21        proc = get_proc();
22    }
23
24    // Recheck: if no process available and loader is done, stop CPU
25    if (proc == NULL && done) {
26        printf("\tCPU %d stopped\n", id);
27        break;
28    } else if (proc == NULL) {
29        next_slot(timer_id);
30        continue;
31    } else if (time_left == 0) {
32        printf("\tCPU %d: Dispatched process %2d\n", id, proc->pid);
33        time_left = time_slot;
34    }
35
36    // Execute one instruction
37    run(proc);
38    time_left--;
39    next_slot(timer_id);
40 }
```

Listing 5: Process Dispatch in cpu_routine (os.c)

This loop implements **Preemptive Scheduling**. A process cannot hold the CPU forever because `time_left` is decremented every cycle. Once it hits zero, the CPU forces a context switch by putting the process back in the queue and fetching the next one.

Process Termination Path

When a process finishes all its instructions (`proc->pc == proc->code->size`), the CPU must safely remove it from the system. Rather than calling `purgequeue()` directly on the `running_list`—which would be unsafe in a multi-threaded environment—the code calls the wrapper function `remove_running_proc(proc)` (defined in `sched.c`). This wrapper acquires the global `queue_lock` mutex before invoking `purgequeue(&running_list, proc)`, ensuring that concurrent CPU threads do not corrupt the list during removal:

```
1 void remove_running_proc(struct pcb_t *proc) {  
2     pthread_mutex_lock(&queue_lock);  
3     purgequeue(&running_list, proc);  
4     pthread_mutex_unlock(&queue_lock);  
5 }
```

Listing 6: Thread-safe process removal (`sched.c`)

2.2 Instruction Execution (`run()`)

The `run()` function decodes the instruction pointed to by the Program Counter (`pc`) and routes it to the corresponding OS subsystem. It returns an `int` status code: 0 indicates successful execution and 1 signals a failure (e.g., invalid register index, denied kernel-mode instruction).

```
1 int run(struct pcb_t *proc) {  
2     if (proc->pc >= proc->code->size) return 1;  
3  
4     struct inst_t ins = proc->code->text[proc->pc];  
5     proc->pc++;  
6     int stat = 1;  
7     switch (ins.opcode) {  
8     case CALC:  
9         stat = calc(proc);  
10        break;  
11    case ALLOC:  
12#ifdef MM_PAGING  
13        stat = liballoc(proc, ins.arg_0, ins.arg_1);  
14#else  
15        stat = alloc(proc, ins.arg_0, ins.arg_1);  
16#endif  
17        break;  
18    case KMALLOC:  
19        if (proc->mode_bit == 1) { stat = 1; break; }  
20    }
```



```
20     stat = libkmem_malloc(proc, ins.arg_0, ins.arg_1);
21     break;
22     case FREE:
23 #ifdef MM_PAGING
24     stat = libfree(proc, ins.arg_0);
25 #else
26     stat = free_data(proc, ins.arg_0);
27 #endif
28     break;
29     case READ:
30 #ifdef MM_PAGING
31     {
32         uint32_t read_val = 0;
33         stat = libread(proc, ins.arg_0, ins.arg_1, &read_val);
34         if (stat == 0) {
35             if (valid_reg_index(proc, ins.arg_2))
36                 proc->regs[ins.arg_2] = read_val;
37             else stat = 1;
38         }
39     }
40 #else
41     stat = read(proc, ins.arg_0, ins.arg_1, ins.arg_2);
42 #endif
43     break;
44     case WRITE:
45 #ifdef MM_PAGING
46     stat = libwrite(proc, ins.arg_0, ins.arg_1, ins.arg_2);
47 #else
48     stat = write(proc, ins.arg_0, ins.arg_1, ins.arg_2);
49 #endif
50     break;
51     case SYSCALL:
52     proc->mode_bit = 0; // Enter Kernel Mode
53     stat = libsyscall(proc, ins.arg_0, ins.arg_1, ins.arg_2, ins.
arg_3);
54     proc->mode_bit = 1; // Return to User Mode
55     break;
56     default:
57     stat = 1;
58 }
59 return stat;
60 }
```

Listing 7: Instruction Routing in cpu.c

Dual-Mode Operation & Mode Bit Transitions

The SYSCALL case in the switch statement illustrates the simulator's dual-mode protection mechanism. Before dispatching the system call, the CPU sets `proc->mode_bit = 0` (Kernel Mode), and after the system call returns, it restores `proc->mode_bit = 1` (User Mode). This sandwich ensures that `libsyscall` and the kernel handler it invokes execute with full kernel privileges, while all other instructions run in restricted user mode.

Crucially, kernel-only instructions (`KMALLOC`, `KMEM_CACHE_CREATE`, `KMEM_CACHE_ALLOC`, `COPY_FROM_USER`, `COPY_TO_USER`) are **guarded** by an explicit `if (proc->mode_bit == 1) { stat = 1; break; }` check. Since processes always start with `mode_bit = 1` (set in `loader.c:59`), these instructions are silently rejected unless the process is currently inside a system call handler. This mirrors the hardware protection ring architecture of real CPUs.

Virtual Registers & Symbol Table

The simulated architecture provides **10 general-purpose registers** per process (`regs[0]–regs[9]`), as defined in `common.h`:

```
1 addr_t regs[10]; // Registers, store address of allocated regions
```

The validation function `valid_reg_index()` ensures instructions don't overflow this array. `CALC` operates entirely in registers. Memory operations like `ALLOC` return virtual addresses stored in these registers for later use by `READ/WRITE`.

Note: The `mm_struct` separately maintains a **symbol table** (`symrgtbl[30]`) with 30 entries (`PAGING_MAX_SYMTBL_SZ = 30`) that maps register indices to full virtual memory region descriptors (`vm_rg_struct`). This is a kernel-side tracking structure, distinct from the 10 CPU registers.

3 Scheduling

3.1 Implementation Breakdown

The scheduler simulates a multi-processing environment where CPU time must be divided fairly among competing processes. It relies on two core modules: `queue.c` for priority-based ready queue management and `sched.c` for the complex scheduling logic.

3.1.1 Ready Queue Management (queue.c)

The ready queues are implemented as separate arrays of `pcb_t` structs, allocating one queue per priority level. This avoids sorting overhead when enqueueing processes.

- `enqueue(q, proc)`: Appends a PCB to the end of the array. The time complexity is $\mathcal{O}(1)$. This ensures that inserting a process into the system is incredibly fast. A guard check ensures the queue does not exceed `MAX_QUEUE_SIZE`.
- `dequeue(q)`: Removes a process from the front (FIFO order). Since it's implemented via arrays rather than linked lists, removing an element requires shifting the remaining elements left, leading to an $\mathcal{O}(n)$ time complexity.
- `purgequeue(q, proc)`: Iterates over the queue to find a specific `pcb_t*` **pointer** (not a PID) and removes it. Internally it uses pointer equality (`q->proc[i] == proc`). It also operates in $\mathcal{O}(n)$ time. PID-based lookup is handled by the separate `find_proc_by_pid()` function in `sched.c`.

3.1.2 Scheduling Logic (sched.c)

The operating system implements a strict **Multi-Level Queue (MLQ)** algorithm, maintaining 140 different priority levels (`MAX_PRIO = 140`).

```
1 static struct queue_t ready_queue; // legacy queue
2 static struct queue_t run_queue;   // legacy run queue (obsoleted)
3 static struct queue_t running_list; // currently dispatched processes
4 static pthread_mutex_t queue_lock;  // synchronization mutex
5
6 #ifdef MLQ_SCHED
7 static struct queue_t mlq_ready_queue[MAX_PRIO];
8 static int slot[MAX_PRIO]; // Remaining CPU slots for each queue
9 #endif
```

Listing 8: Scheduler Definitions

The slot Budgeting System

Unlike simple Priority Scheduling which causes severe starvation for low-priority processes, our MLQ policy allocates a specific slot (time budget) to each priority queue. The number of slots is calculated as $\text{MAX_PRIO} - \text{prio}$. A queue can only dispatch processes until its slots run out. Once empty, it is locked out until all other non-empty queues also deplete their slots, forcing the system to give lower-priority queues a chance to execute.

Example: Priority 0 (highest) receives $140 - 0 = 140$ slots. Priority 139 (lowest) receives $140 - 139 = 1$ slot. This ensures priority 0 gets 140 times more CPU opportunities per cycle than priority 139, but priority 139 is *never starved* entirely.

The logic revolves around three core functions:

- `add_mlq_proc(proc)`: Inserts a newly created process directly into its respective priority queue (`mlq_ready_queue[proc->prio]`). Since this process has just been loaded by the system and has never been executed, there is no need to check or remove it from the `running_list`. Before enqueueing, it sets up the kernel structure pointers (`proc->krnl->ready_queue`, `proc->krnl->mlq_ready_queue`, `proc->krnl->running_list`) so that later system call handlers can access the scheduler queues.
- `get_mlq_proc()`: Scans from priority 0 (highest) to 139 (lowest). It finds the highest priority queue that is both non-empty and has `slot > 0`. It dequeues a process, decrements the slot counter, and adds it to the `running_list`.
- `put_mlq_proc(proc)`: When a process uses up its time slice, the CPU interrupts it. This function removes the process from `running_list` and pushes it to the back of its priority queue to maintain Round-Robin fairness. It also refreshes the kernel structure pointers, identically to `add_mlq_proc`.

3.2 Code Walkthrough: `get_mlq_proc()`

Let's analyze the precise code that fetches the next process.

```
1 struct pcb_t *get_mlq_proc(void)
2 {
3     struct pcb_t *proc = NULL;
4
5     pthread_mutex_lock(&queue_lock);
6     {
7         int i;
```

```
8  int has_ready = 0;
9  int selected_prio = -1;
10
11  for (i = 0; i < MAX_PRIO; i++) {
12      if (!empty(&mlq_ready_queue[i])) {
13          has_ready = 1;
14          if (slot[i] > 0) {
15              selected_prio = i;
16              break;
17          }
18      }
19  }
20
21  if (selected_prio == -1 && has_ready) {
22      for (i = 0; i < MAX_PRIO; i++)
23          slot[i] = MAX_PRIO - i;
24
25      for (i = 0; i < MAX_PRIO; i++) {
26          if (!empty(&mlq_ready_queue[i])) {
27              selected_prio = i;
28              break;
29          }
30      }
31  }
32
33  if (selected_prio != -1) {
34      proc = dequeue(&mlq_ready_queue[selected_prio]);
35      slot[selected_prio]--;
36      enqueue(&running_list, proc);
37  }
38  }
39  pthread_mutex_unlock(&queue_lock);
40
41  return proc;
42 }
```

Listing 9: get_mlq_proc Implementation

Analysis: The outer loop checks from $i = 0$ upwards. It finds the first queue that has processes and still has a positive $\text{slot}[i]$. The flag `has_ready` tracks whether any queue has waiting processes. Crucially, if `selected_prio` remains -1 but `has_ready` is true, it means all active queues have exhausted their slot budgets. The scheduler immediately resets every slot back to $\text{MAX_PRIO} - i$ and retries the fetch. This reset cycle is the mechanism that guarantees starvation prevention.

3.3 Kernel Structure Pointer Setup

Both `add_mlq_proc` and `put_mlq_proc` configure the process's kernel structure pointers before enqueueing. This is critical because system call handlers (e.g., `__sys_memmap`) need access to the scheduler's queues to locate processes by PID:

```
1 void put_mlq_proc(struct pcb_t *proc) {  
2     proc->krnl->ready_queue = &ready_queue;  
3     proc->krnl->mlq_ready_queue = mlq_ready_queue;  
4     proc->krnl->running_list = &running_list;  
5  
6     pthread_mutex_lock(&queue_lock);  
7     purgequeue(&running_list, proc);  
8     enqueue(&mlq_ready_queue[proc->prio], proc);  
9     pthread_mutex_unlock(&queue_lock);  
10 }
```

Listing 10: Kernel pointer configuration in `put_mlq_proc`

Without this step, the `find_proc_by_pid()` function called from within kernel-mode system calls would not be able to traverse the scheduler queues to resolve a process PID to its `pcb_t` structure.

3.4 Thread Safety

Since this OS supports multiple simulated CPUs concurrently fetching and putting processes, every queue modification is enclosed in `pthread_mutex_lock(&queue_lock)` and `pthread_mutex_unlock(&queue_lock)`. This prevents race conditions where two CPUs might dequeue the exact same `pcb_t` block, leading to catastrophic kernel crashes.

3.5 Gantt Diagram

By running the system against `sched.input`, `sched_0.input`, and `sched_1.input`, we observe the execution flow across the available virtual CPUs.

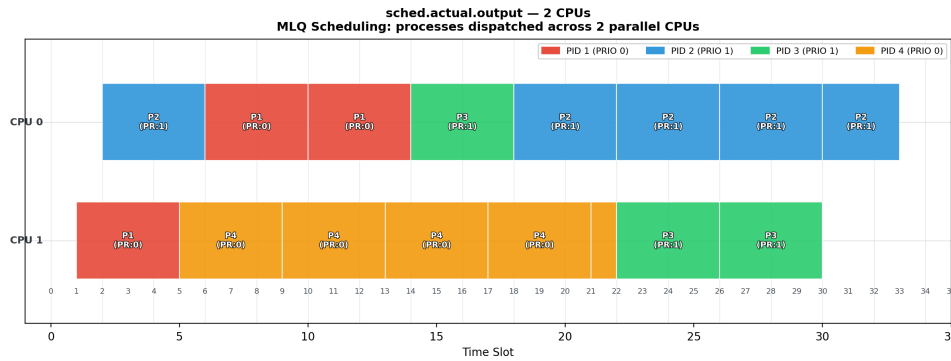


Figure 1: Gantt Diagram Execution Trace of sched.input (2 CPUs)

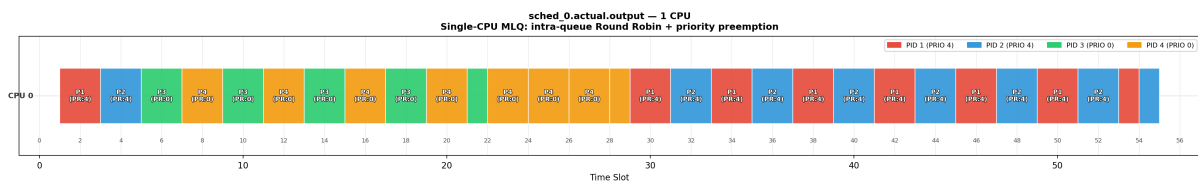


Figure 2: Gantt Diagram Execution Trace of sched_0.input (1 CPU)

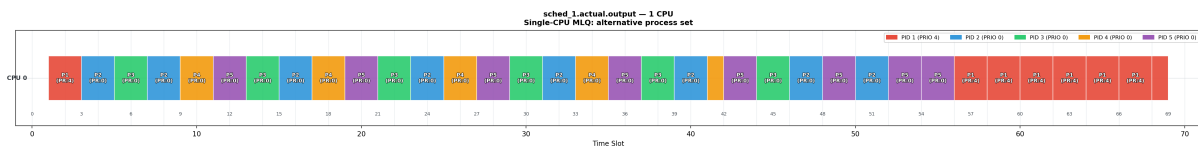


Figure 3: Gantt Diagram Execution Trace of sched_1.input (1 CPU)

4 Memory Management

4.1 Virtual Memory Architecture

The memory manager utilizes a paging scheme to separate the physical memory (RAM and SWAP) from the contiguous virtual addresses seen by user processes.

The virtual memory subsystem tracks allocations at three levels:

1. **Virtual Memory Areas (vm_area_struct):** Represents large contiguous chunks of virtual memory. Each area maintains a sbrk pointer representing the top of the heap. Memory beneath sbrk is managed by vm_freerg_list, which keeps track of freed regions to combat internal fragmentation.
2. **Symbol Table (symrgtbl):** Since processes execute simple instructions via registers, this table maps register indices (0 to 29) to allocated virtual memory regions.

Each entry is a `vm_rg_struct` with `rg_start` and `rg_end` fields defining the byte range. The table size is fixed at `PAGING_MAX_SYMTBL_SZ = 30`.

3. **Page Tables:** Translates virtual pages to physical frames. In the 64-bit build (MM64), this is a 5-level hierarchy (`PGD → P4D → PUD → PMD → PT`). In the legacy 32-bit build, a single flat `pgd` array is used.

4.2 The ALLOC Operation (`liballoc`)

When a user process issues an `alloc [size] [reg]` instruction, `liballoc()` handles it via a highly optimized Two-Phase Allocation Strategy.

Two-Phase Allocation Strategy

Phase 1: Memory Reuse (First-Fit Algorithm)

Before requesting new pages from the kernel, the system scans `vm_freerg_list` via `get_free_vmrg_area()`. If a previously freed block is large enough, it assigns the block to the requested register, avoiding expensive kernel system calls. The first-fit traversal splits the chosen region: the requested portion is assigned to the symbol table, and any leftover space remains in the free list.

Phase 2: Heap Extension (System Call)

If no free blocks are suitable, the system invokes `SYSMEM_INC_OP` via `_syscall(caller->krnl, caller->pid, 17, ®s)` to ask the kernel to increase `sbrk`. The size is page-aligned using `PAGING_PAGE_ALIGNSZ(size)` (padded to the next multiple of page size). The kernel handler `inc_vma_limit()` expands the VMA boundary, calls `vm_map_ram()` to allocate physical frames and populate page table entries, and advances `sbrk`. Leftover space from the page-aligned expansion is added back to `vm_freerg_list` for future first-fit allocations.

```

1 int __alloc(struct pcb_t *caller, int vmaid, int rgid,
2             addr_t size, addr_t *alloc_addr)
3 {
4     pthread_mutex_lock(&mmvm_lock);
5     struct vm_area_struct *cur_vma = get_vma_by_num(caller->krnl->mm,
6             vmaid);
7
8     // Phase 1: Try to reuse a freed region
9     if (get_free_vmrg_area(caller, vmaid, size, &rgnode) == 0) {
10         caller->krnl->mm->symrgtbl[rgid].rg_start = rgnode.rg_start;
11         caller->krnl->mm->symrgtbl[rgid].rg_end = rgnode.rg_end;
12         *alloc_addr = rgnode.rg_start;
13         pthread_mutex_unlock(&mmvm_lock);

```



```
13     return 0;
14 }
15
16 // Phase 2: Expand the heap via system call
17 int old_sbrk = cur_vma->sbrk;
18 struct sc_regs regs;
19 regs.a1 = SYSMEM_INC_OP;
20 regs.a2 = vmaid;
21 regs.a3 = PAGING_PAGE_ALIGNSZ(size);
22 _syscall(caller->krnl, caller->pid, 17, &regs);
23
24 // Register the expanded region in the symbol table
25 caller->krnl->mm->symrgtbl[rgid].rg_start = old_sbrk;
26 caller->krnl->mm->symrgtbl[rgid].rg_end = old_sbrk + size;
27 *alloc_addr = old_sbrk;
28
29 pthread_mutex_unlock(&mmvm_lock);
30 return 0;
31 }
```

Listing 11: Heap Extension via _syscall (libmem.c)

4.3 The FREE Operation (__free)

The free [reg] instruction is non-destructive to the physical page table.

```
1 int __free(struct pcb_t *caller, int vmaid, int rgid) {
2     pthread_mutex_lock(&mmvm_lock);
3     struct vm_rg_struct *rgnode = get_symrg_byid(caller->krnl->mm, rgid
4 );
5
6     if (rgnode->rg_start == 0 && rgnode->rg_end == 0) {
7         pthread_mutex_unlock(&mmvm_lock);
8         return -1; // Nothing to free (never allocated)
9     }
10
11     // Copy the region to a new free node
12     struct vm_rg_struct *freerg_node = malloc(sizeof(struct
13 vm_rg_struct));
14     freerg_node->rg_start = rgnode->rg_start;
15     freerg_node->rg_end = rgnode->rg_end;
16     freerg_node->rg_next = NULL;
17
18     // Erase from symbol table and add to free list
19     rgnode->rg_start = rgnode->rg_end = 0;
```

```
18     rgnode->rg_next = NULL;
19     enlist_vm_freerg_list(caller->krnl->mm, freerg_node);
20     pthread_mutex_unlock(&mmvm_lock);
21     return 0;
22 }
```

Listing 12: FREE implementation (libmem.c)

Analysis: By choosing not to deallocate the physical frames immediately, the OS significantly reduces the overhead of page table manipulation. Re-allocating the same space later instantly succeeds via Phase 1. Note the guard check `rg_start == 0 && rg_end == 0`: if a register was never allocated (e.g., a blocked `KMALLOC` instruction), the free operation returns `-1`. This produces the `libfree:218` failed message seen in the execution traces.

4.4 READ/WRITE & Page Replacement Strategy

Reads and Writes are handled by `__read` and `__write`. These traverse the virtual bounds to find the exact byte.

```
1  int pg_getval(struct mm_struct *mm, int addr, BYTE *data, struct pcb_t
    *caller) {
2      int pgn = PAGING_PGN(addr);
3      int off = PAGING_OFFST(addr);
4      int fpn;
5
6      /* Get the page to RAM. Handles page faults internally! */
7      if (pg_getpage(mm, pgn, &fpn, caller) != 0)
8          return -1;
9
10     int phyaddr = (fpn << PAGING_ADDR_FPN_LOBIT) + off;
11
12     struct sc_regs regs;
13     regs.a1 = SYSMEM_IO_READ;
14     regs.a2 = phyaddr;
15     regs.a3 = 0;
16     _syscall(caller->krnl, caller->pid, 17, &regs);
17     *data = (BYTE)regs.a3;
18     return 0;
19 }
```

Listing 13: Reading Virtual Memory (libmem.c)

Physical Address Computation

The physical address is computed as $(\text{fpn} \ll \text{PAGING_ADDR_FPN_LOBIT}) + \text{off}$, which is equivalent to $\text{fpn} * \text{PAGING_PAGESZ} + \text{off}$. The bit-shift formulation is used because $\text{PAGING_ADDR_FPN_LOBIT}$ equals $\log_2(\text{PAGING_PAGESZ})$, and left-shifting by this value is a faster equivalent of multiplication by the page size. The read/write operations themselves are performed through the system call interface (`SYSMEM_IO_READ`/`SYSMEM_IO_WRITE`) to maintain the user-kernel boundary: the actual `MEMPHY_read`/`MEMPHY_write` calls happen inside `__sys_mmap` in kernel space.

4.5 Detailed Page Fault Mechanics

If the page resides in SWAP instead of RAM, `pg_getpage()` triggers a **Page Fault**. The kernel intercepts this and initiates a Page Replacement sequence using a strict FIFO queue (`mm->fifo_pgn`):

Case 1: Free frame available. If `MEMPHY_get_freefp()` succeeds, the page is assigned the free frame directly via `pte_set_fpn()` and added to the FIFO list.

Case 2: No free frames (full replacement path):

1. **Find Victim:** Dequeue the oldest page from `fifo_pgn` using `find_victim_page()`. This function walks the singly-linked list to find the **tail node**—the oldest entry—implementing true FIFO ordering.
2. **Swap Out:** Copy the victim page from RAM into the secondary SWAP device using `__swap_cp_page(mram, vicfpn, active_mswp, swpfpn)`. Update the victim's PTE via `pte_set_swap()` to record the swap device ID and swap offset.
3. **Swap In:** If the requested page was previously swapped out (indicated by the `PAGING_PTE_SWAPPED_MASK` bit), copy it back from SWAP to the victim's freed frame. Otherwise, zero-fill the frame (new allocation).
4. **Update PTEs:** Update the Page Table Entry for the newly loaded page via `pte_set_fpn()` to clear the swapped bit and set the FPN. Push the newly loaded page into `fifo_pgn`.

Example of a Page Fault:

Assume RAM has exactly 2 frames. Process 1 requests 3 pages. 1. CPU executes `alloc 512` (Page 0 and Page 1). RAM Frame 0 and 1 are used. 2. CPU executes `alloc 256` (Page 2). RAM is full. The OS triggers a Page Replacement (Swapping) sequence. 3.

The OS selects Page 0 (oldest) as the victim. 4. `__swap_cp_page(RAM, 0, SWAP, 0)` is called. Page 0 is now on disk. 5. RAM Frame 0 is now assigned to Page 2. 6. If the CPU subsequently tries to read Page 0, a *second* Page Fault occurs. The OS will select Page 1 as the victim, swap it to disk, and bring Page 0 back into RAM Frame 1.

4.6 Kernel Memory Operations

The specification defines three kernel-space memory instructions that enable privileged allocation patterns modeled after Linux's kernel memory subsystem. These are implemented in `libmem.c` and protected by the `mode_bit` check described in Section 2.

4.6.1 KMALLOC — Physically Contiguous Allocation

The `kmalloc [size] [reg]` instruction allocates a contiguous block of *physical* memory. Unlike user-space `ALLOC` which allows the OS to scatter pages across non-contiguous frames, `KMALLOC` requires that the allocated frames form a **consecutive sequence** in physical RAM:

```
1 addr_t __kmalloc(struct pcb_t *caller, int vmaid, int rgid,
2                 addr_t size, addr_t *alloc_addr)
3 {
4     // First, attempt a normal virtual allocation
5     if (__alloc(caller, 0, rgid, size, alloc_addr) < 0)
6         return (addr_t)-1;
7
8     // Then verify physical contiguity
9     if (!is_phys_contiguous_region(caller, *alloc_addr, size)) {
10         __free(caller, 0, rgid); // Roll back
11         return (addr_t)-1;
12     }
13     return *alloc_addr;
14 }
```

Listing 14: `__kmalloc` contiguity check (`libmem.c`)

The helper `is_phys_contiguous_region()` walks the page table entries for the allocated virtual range and verifies that each successive PTE's frame page number (FPN) is exactly one greater than the previous. If the frames are scattered, the allocation is rolled back. This models real-world DMA buffers and kernel data structures that require contiguous physical memory.

4.6.2 User/Kernel Address Tagging

To distinguish user-space allocations from kernel-space allocations, the system uses a **tag bit** mechanism:

```
1 #ifdef MM64
2 #define KERNEL_ADDR_TAG ((addr_t)1ULL << 63)
3 #else
4 #define KERNEL_ADDR_TAG ((addr_t)1U << 31)
5 #endif
6
7 static int is_kernel_addr(addr_t addr) {
8     return (addr & KERNEL_ADDR_TAG) != 0;
9 }
10 static addr_t to_kernel_addr(addr_t addr) {
11     return addr | KERNEL_ADDR_TAG;
12 }
```

Listing 15: Address tagging (libmem.c)

When `libkmem_malloc()` stores a kernel allocation address in a register, it tags the address by setting the most-significant bit (bit 63 for 64-bit, bit 31 for 32-bit): `caller->regs[reg_index] = to_kernel_addr(addr)`. Subsequent `libread/libwrite` calls check `is_kernel_addr()` and **reject** user-mode accesses to kernel-tagged addresses. This mirrors the canonical address split in real x86-64 architectures, where user-space addresses have their top bits clear and kernel-space addresses have them set.

4.6.3 Slab Cache: `KMEM_CACHE_CREATE` & `KMEM_CACHE_ALLOC`

The slab allocator avoids repeated allocation/initialization overhead by pre-defining fixed-size object pools:

```
1 int libkmem_cache_pool_create(struct pcb_t *caller, uint32_t size,
2                               uint32_t align, uint32_t cache_pool_id)
3 {
4     if (caller->krnl->mm->kcpooltbl == NULL)
5         caller->krnl->mm->kcpooltbl = calloc(PAGING_MAX_SYMTBL_SZ,
6                                              sizeof(struct
7                                              kcache_pool_struct));
8
9     if (cache_pool_id >= PAGING_MAX_SYMTBL_SZ)
10         return -1;
11
12     caller->krnl->mm->kcpooltbl[cache_pool_id].size = size;
13     caller->krnl->mm->kcpooltbl[cache_pool_id].align = align;
14     caller->krnl->mm->kcpooltbl[cache_pool_id].storage = 0;
```

```
14 return 0;  
15 }
```

Listing 16: Cache pool creation (libmem.c)

KMEM_CACHE_CREATE registers a cache pool with a predefined object size and alignment. When KMEM_CACHE_ALLOC is later called, it delegates to `__kmalloc` using the pool's stored size, guaranteeing that every allocation from the same cache pool produces physically contiguous blocks of identical size. This mirrors the Linux kernel's `kmem_cache_create()/kmem_cache_alloc()` API.

4.6.4 COPY_FROM_USER & COPY_TO_USER

These instructions enable safe data transfer across the user/kernel boundary:

```
1 int libkmem_copy_from_user(struct pcb_t *caller, uint32_t source,  
2                           uint32_t destination, uint32_t offset,  
3                           uint32_t size) {  
4     uint32_t i;  
5     BYTE data;  
6     for (i = 0; i < size; i++) {  
7         if (__read_user_mem(caller, 0, source, offset + i, &data) < 0)  
8             return -1;  
9         if (__write_kernel_mem(caller, 0, destination, i, data) < 0)  
10            return -1;  
11     }  
12     return 0;  
13 }
```

Listing 17: Cross-boundary memory copy (libmem.c)

COPY_FROM_USER reads byte-by-byte from a user-space region and writes to a kernel-space region. COPY_TO_USER performs the reverse. The helper functions `__read_user_mem` and `__write_kernel_mem` enforce address-space isolation: `__read_user_mem` rejects reads from kernel-tagged addresses (via `is_kernel_addr()`), and `__write_kernel_mem` rejects writes to non-kernel-tagged addresses. This prevents user processes from smuggling kernel pointers to escalate privileges.

5 System Call Integration

A robust OS must isolate kernel memory from user processes. The `mmap` interface (Syscall number 17, dispatched via `libsyscall`) guarantees this protection.

Terminology Clarification: Opcode vs. Syscall Number

In this simulator, SYSCALL is an **instruction opcode** (defined in `common.h`'s `enum ins_opcode_t`). When `cpu.c:run()` encounters this opcode, it calls `libsyscall(proc, arg_0, arg_1, arg_2, arg_3)` and sandwiches the call between `mode_bit = 0` (Kernel) and `mode_bit = 1` (User) transitions. Inside `libsyscall`, argument `arg_0` carries the **syscall number** (e.g., 17 for `__sys_mmap`), which is then dispatched through `syscalltbl.lst`. The two concepts — the opcode SYSCALL and the syscall number 17 — are distinct.

5.1 Register-Based Argument Passing

Arguments are packed into a `sc_regs` structure. Register `a1` contains the operation code, while `a2` and `a3` hold arguments.

Opcode	Kernel Action
SYSTEM_MAP_OP (1)	Executes <code>vmap_pgd_memset()</code> to emulate page directory behavior.
SYSTEM_INC_OP (2)	Executes <code>inc_vma_limit()</code> to safely expand the heap.
SYSTEM_SWP_OP (3)	Executes <code>__mm_swap_page()</code> for disk swapping.
SYSTEM_IO_READ (4)	Reads exactly 1 byte from physical memory.
SYSTEM_IO_WRITE (5)	Writes exactly 1 byte to physical memory.

Table 1: SYSCALL 17 Operations

5.2 System Call Dispatching and Protection

The kernel provides a centralized dispatcher that validates every request. The function `_syscall()` (with a leading underscore, defined in `syscall.c`) uses C preprocessor X-macros to dynamically build its dispatch switch statement from the `syscalltbl.lst` file:

```

1 #define __SYSCALL(nr, sym) case nr: return __##sym(krnl, pid, regs);
2 int _syscall(struct krnl_t *krnl, uint32_t pid, uint32_t nr,
3             struct sc_regs* regs)
4 {

```

```
5     switch (nr) {
6         #include "syscalltbl.lst"
7         default: return __sys_ni_syscall(krnl, regs);
8     }
9 };
10
11 // syscalltbl.lst contains:
12 // __SYSCALL(0, sys_listsyscall)
13 // __SYSCALL(17, sys_mmap)
```

Listing 18: Kernel Syscall Dispatcher (syscall.c)

The X-macro `__SYSCALL(nr, sym)` expands each line of `syscalltbl.lst` into a case statement. For example, `__SYSCALL(17, sys_mmap)` becomes case 17: `return __sys_mmap(krnl, pid, regs);`. The `__sys_ni_syscall` default handler safely ignores invalid syscall numbers.

5.3 PID-Based Kernel Traversal

The specification explicitly requires: *“The process PCB is prohibited from being passed directly. It must be accessed by traversing in kernel mode to obtain the PCB from the kernel structure using the given PID.”* The system call handler `__sys_mmap()` in `sys_mem.c` adheres to this requirement:

```
1 int __sys_mmap(struct krnl_t *krnl, uint32_t pid, struct sc_regs*
   regs) {
2     int memop = regs->a1;
3     BYTE value;
4
5     /* Resolve caller by PID -- NOT by passing pcb_t* directly */
6     struct pcb_t *caller = find_proc_by_pid(pid);
7     if (caller == NULL) return -1;
8
9     switch (memop) {
10        case SYSMEM_MAP_OP:  vmap_pgd_memset(caller, regs->a2, regs->a3);
11        break;
12        case SYSMEM_INC_OP:  inc_vma_limit(caller, regs->a2, regs->a3);
13        break;
14        case SYSMEM_SWP_OP:  __mm_swap_page(caller, regs->a2, regs->a3);
15        break;
16        case SYSMEM_IO_READ:
17            MEMPHY_read(krnl->mram, regs->a2, &value);
18            regs->a3 = value;
19            break;
```



```
17     case SYSEMIO_WRITE:
18         MEMPHY_write(krnl->mram, regs->a2, regs->a3);
19         break;
20     }
21     return 0;
22 }
```

Listing 19: PID-based process lookup in `__sys_mmap` (`sys_mem.c`)

The `find_proc_by_pid()` function (in `sched.c`) traverses two data structures under the global `queue_lock` mutex:

1. **The `running_list`:** Searched first, since the caller is most likely currently running.
2. **All 140 MLQ ready queues:** Searched sequentially if the process is not found in the running list (possible during concurrent scheduling transitions).

This PID-based lookup ensures that the system call handler operates entirely in kernel space, using only the PID as a numeric handle. The user process never passes its `pcb_t*` pointer across the user/kernel boundary, preventing forged pointer attacks.

5.4 Wrapper Library (`libstd.c`)

The user-space library `libsyscall()` in `libstd.c` acts as a thin wrapper that packs arguments into a `sc_regs` structure and invokes the kernel dispatcher:

```
1 int libsyscall(struct pcb_t *caller,
2               uint32_t syscall_idx,
3               arg_t a1, arg_t a2, arg_t a3)
4 {
5     struct sc_regs regs;
6     regs.a1 = a1;
7     regs.a2 = a2;
8     regs.a3 = a3;
9     return _syscall(caller->krnl, caller->pid, syscall_idx, &regs);
10 }
```

Listing 20: User-space syscall wrapper (`libstd.c`)

Protection and Isolation: Without system calls, a process could manually manipulate its `struct pcb_t` and page tables to read another user's data or crash the system. By forcing the user to call `syscall(17)`, the CPU switches to Kernel Mode and resolves the caller by PID. The `SYSEMIO_READ` and `SYSEMIO_WRITE` operations access physical memory through `krnl->mram`—the kernel's own memory reference—rather than any

user-provided pointer. Argument validation and alignment handling are then enforced in downstream memory-management routines (for example, `inc_vma_limit()` and related VM helpers).

6 Multi-level Paging (64-bit Scheme)

To handle the massive 128 PiB virtual address space associated with a 64-bit architecture, the standard single-level page table is highly inefficient. We implemented a 5-level page table scheme in `mm64.c`.

6.1 Address Translation Process

The 64-bit virtual address is segmented using strict bitmasks:

```
1 int get_pd_from_address(addr_t addr, addr_t* pgd, addr_t* p4d, addr_t*
2   pud, addr_t* pmd, addr_t* pt) {
3     *pgd = (addr & PAGING64_ADDR_PGD_MASK) >> PAGING64_ADDR_PGD_LOBIT;
4     // bits 48-56
5     *p4d = (addr & PAGING64_ADDR_P4D_MASK) >> PAGING64_ADDR_P4D_LOBIT;
6     // bits 39-47
7     *pud = (addr & PAGING64_ADDR_PUD_MASK) >> PAGING64_ADDR_PUD_LOBIT;
8     // bits 30-38
9     *pmd = (addr & PAGING64_ADDR_PMD_MASK) >> PAGING64_ADDR_PMD_LOBIT;
10    // bits 21-29
11    *pt = (addr & PAGING64_ADDR_PT_MASK) >> PAGING64_ADDR_PT_LOBIT;
12    // bits 12-20
13
14    if (((addr >> PAGING64_ADDR_PGD_HIBIT) & 0x1) != 0) {
15        /* Kernel-space canonical address: high bits must stay 1 */
16        *pgd |= BIT_ULL(PAGING64_ADDR_PGD_HIBIT -
17        PAGING64_ADDR_PGD_LOBIT);
18    }
19    return 0;
20 }
```

Listing 21: 5-Level Address Bit Extraction (`mm64.c`)

Given a user-space virtual address of `0x0000000000400000` (4MB), the extraction process yields the following indices:

- **PGD Index:** $(0x400000 \gg 48) \& 0x1FF = 0$
- **P4D Index:** $(0x400000 \gg 39) \& 0x1FF = 0$
- **PUD Index:** $(0x400000 \gg 30) \& 0x1FF = 0$

- **PMD Index:** $(0x400000 \gg 21) \& 0x1FF = 2$
- **PT Index:** $(0x400000 \gg 12) \& 0x1FF = 0$

The virtual address $0x0000000000400000$ is translated through

$$\text{PGD}[0] \rightarrow \text{P4D}[0] \rightarrow \text{PUD}[0] \rightarrow \text{PMD}[2] \rightarrow \text{PT}[0]$$

6.2 Canonical Address Handling

In a 64-bit architecture, the hardware typically implements only 48-bit or 57-bit addressing rather than the full 64-bit range. To prevent software from misusing the unused higher-order bits for non-address purposes, the architecture enforces **canonical addresses**: all unused bits must be copies of the highest implemented bit (sign-extension).

In our implementation, `get_pd_from_address()` checks bit 56 (the `PAGING64_ADDR_PGD_HIBIT`). If this bit is set, the address belongs to **kernel space**, and the PGD index is OR-ed with an extra high bit to push it into the upper half of the PGD array. This creates a clean split:

- **User-space addresses** (bits 63–57 all zero): PGD indices 0 to ~ 255
- **Kernel-space addresses** (bits 63–57 all ones): PGD indices 256+

This mirrors Linux's 5-level paging layout where the canonical address hole between $0x00ffffffffffff$ and $0xff00000000000000$ is unmappable, ensuring user processes cannot access kernel memory through arithmetic overflow.

6.3 PTE Bit Layout

Each Page Table Entry (PTE) stores essential metadata, defined by bitmask macros in `mm64.h`:

- **Present bit** (`PAGING_PTE_PRESENT_MASK`): Set to 1 if the page is in RAM; 0 means a page fault is required on access.
- **Swapped bit** (`PAGING_PTE_SWAPPED_MASK`): Set to 1 if the page data currently resides on the SWAP device.
- **Dirty bit** (`PAGING_PTE_DIRTY_MASK`): Set to 1 if the page has been written since it was last loaded from swap.

- **Frame Page Number (PAGING_PTE_FPN_MASK, low bits):** The actual Physical Frame Number assigned to this virtual page.
- **Swap type/offset (PAGING_PTE_SWPTYP_MASK / PAGING_PTE_SWPOFF_MASK):** When the page is swapped out, these fields record which swap device and at which offset the page data resides.

6.4 Hierarchical Entry Population

```
1 int pte_set_fpn(struct pcb_t *caller, addr_t pgn, addr_t fpn) {
2     addr_t pgd=0, p4d=0, pud=0, pmd=0, pt=0;
3     struct krnl_t *krnl = caller->krnl;
4
5     get_pd_from_pagenum(pgn, &pgd, &p4d, &pud, &pmd, &pt);
6
7     /* Set intermediate directory entries as present */
8     init_pte(&krnl->mm->pgd[pgd % PAGING64_MAX_PGN], 1, p4d + 1, 0, 0,
9     0, 0);
10    init_pte(&krnl->mm->p4d[p4d % PAGING64_MAX_PGN], 1, pud + 1, 0, 0,
11    0, 0);
12    init_pte(&krnl->mm->pud[pud % PAGING64_MAX_PGN], 1, pmd + 1, 0, 0,
13    0, 0);
14    init_pte(&krnl->mm->pmd[pmd % PAGING64_MAX_PGN], 1, pt + 1, 0, 0,
15    0, 0);
16
17    addr_t *pte = &krnl->mm->pt[pgn % PAGING64_MAX_PGN];
18
19    SETBIT(*pte, PAGING_PTE_PRESENT_MASK);
20    CLRBIT(*pte, PAGING_PTE_SWAPPED_MASK);
21    SETVAL(*pte, fpn, PAGING_PTE_FPN_MASK, PAGING_PTE_FPN_LOBIT);
22    return 0;
23 }
```

Listing 22: Setting a PTE with 5-level hierarchy (mm64.c)

Design Decision: Pre-Allocated vs. Sparse Page Tables

Production operating systems such as Linux implement true sparse hierarchical page tables: lower-level directories (P4D, PUD, PMD, PT) are allocated on demand via `get_free_page()` only when a virtual address in that range is first accessed. This means a process using only 1 MB of heap allocates a tiny fraction of the theoretical page table space.

Our simulator adopts a different trade-off. In `init_mm()` (`mm64.c:447-451`), all five paging-level arrays are **pre-allocated as fixed-size flat arrays** using `calloc(PAGING64_MAX_PGN, sizeof(addr_t))`:

```
1 mm->pgd = calloc(PAGING64_MAX_PGN, sizeof(addr_t));
2 mm->p4d = calloc(PAGING64_MAX_PGN, sizeof(addr_t));
3 mm->pud = calloc(PAGING64_MAX_PGN, sizeof(addr_t));
4 mm->pmd = calloc(PAGING64_MAX_PGN, sizeof(addr_t));
5 mm->pt = calloc(PAGING64_MAX_PGN, sizeof(addr_t));
```

This sacrifices memory efficiency in favor of simplicity and direct array-based indexing ($O(1)$ access via modular arithmetic). The `pte_set_fpn` function still populates the hierarchy chain *logically* by marking `PGD→P4D→PUD→PMD` entries as present only when pages are mapped, demonstrating the correctness of the multi-level translation mechanism.

6.5 The `vmap_pgd_memset` System Call

The specification requires students to implement a `vmap_pgd_memset` operation to emulate page directory behavior for the 64-bit address scheme. This operation is invoked through `SYSEMEM_MAP_OP` (syscall 17, operation code 1):

```
1 int vmap_pgd_memset(struct pcb_t *caller, addr_t addr, int pgnum) {
2     int pgit;
3     addr_t pgn = (addr >> PAGING64_ADDR_PT_SHIFT);
4     addr_t pattern = 0xdeadbeefUL;
5
6     for (pgit = 0; pgit < pgnum; pgit++) {
7         caller->krnl->mm->pt[(pgn + pgit) % PAGING64_MAX_PGN] = pattern
8     };
9     return 0;
10 }
```

Listing 23: `vmap_pgd_memset` (`mm64.c`)

This function fills `pgnum` consecutive page table entries with the sentinel pattern

0xdeadbeef. This serves as a dummy allocation used to test and verify that the 64-bit address scheme's page directory traversal is functioning correctly without performing real physical memory allocation.

6.6 Quantitative Statistics (Required by Specification)

The specification asks for statistics on memory access activity and multi-level paging storage size. We report both here.

1) Multi-level paging storage size

From `mm64.h`,

$$\text{PAGING64_MAX_PGN} = \left\lceil \frac{2^{21}}{4096} \right\rceil = 512$$

Each paging level array stores 512 entries of type `addr_t` (8 bytes on 64-bit):

$$512 \times 8 = 4096 \text{ bytes per level}$$

With 5 levels (PGD, P4D, PUD, PMD, PT):

$$5 \times 4096 = 20480 \text{ bytes} = 20 \text{ KiB per process mm}$$

This matches the `calloc(PAGING64_MAX_PGN, sizeof(addr_t))` allocation pattern in `init_mm()`.

2) Memory access activity statistics from actual outputs

We use trace-observable memory-management events (allocation/free logs and page-table dumps) from provided outputs:

Output file	alloc	free	free failed	pgtbl
<code>os_1_mlq_paging.actual.output</code>	9	6	0	23
<code>os_2_mlq_paging.actual.output</code>	4	4	4	12

Table 2: Trace-derived memory activity statistics from provided testcase outputs.

3) Translation memory-access cost (analytical)

For 5-level paging without TLB hit, a data access requires up to 5 page-walk reads plus 1 data memory access:

$$A_{\text{no TLB}} = 5 + 1 = 6$$

On a TLB hit, only the final data access is required:

$$A_{\text{TLB hit}} = 1$$

These figures explain why hierarchical paging is scalable in address-space size but still benefits strongly from TLB locality.

7 Questionnaire & Practical Examples

Question 1

What are the advantages and disadvantages of using the unified system call interface for manipulating different system components (read/write/free for files, memory, and I/O devices)? Consider how this abstraction influences OS design, performance trade-offs, error handling complexity, and the balance between portability and efficiency.

Answer:

The Unix philosophy of “*everything is a file*” abstracts all I/O resources (disks, sockets, pipes, terminals, memory-mapped devices) into a uniform file descriptor handle. Every resource is manipulated via a small set of system calls: `open()`, `read()`, `write()`, `close()`.

Internal Architecture (VFS Layer):

The kernel implements this via the **Virtual File System (VFS)**, a software indirection layer that maps each syscall to a driver-specific handler through a table of function pointers (`struct file_operations`). When a user calls `write(fd, buf, n)`, the VFS dispatches to the correct handler—whether that is a hard disk driver, a network socket, or a GPU device—without the user knowing which:

```
1 struct file_operations {
2     ssize_t (*read)(struct file *, char __user *, size_t, loff_t *);
3     ssize_t (*write)(struct file *, const char __user *, size_t, loff_t
4         *);
5     int      (*open)(struct inode *, struct file *);
6     int      (*release)(struct inode *, struct file *);
7 };
```

- **Advantages:**

- *Portability & Composability:* A utility like `cat` that reads `stdin` works identically whether `stdin` is a file, a pipe, or a network socket. No code changes are needed.
- *Unified Security Model:* POSIX permission bits (`rwxr-xr-x`) and DAC/MAC policies apply uniformly to all resources through a single mechanism.

- *Simplified OS Design*: The kernel only needs one generic buffer-cache subsystem, one locking strategy, and one error-propagation pathway to cover all device types.
- *In our OS*: The instruction dispatcher in `cpu.c` handles `READ/WRITE/FREE` opcodes directly, while the `SYSCALL` opcode transitions to kernel mode and routes through `libsyscall()` and `_syscall()` (syscall table in `syscall.c`). The unification point is therefore the syscall layer, not a per-opcode table in `cpu.c`.

• Disadvantages:

- *Performance Overhead*: Every access passes through VFS indirection layers. A direct memory-mapped I/O (`mmap`) bypasses this cost entirely, which is why high-performance databases (e.g., PostgreSQL) prefer `mmap` over `read()`.
- *Imperfect Abstractions*: Network devices require connection state, asynchronous events, and specialized options that cannot fit cleanly into `read/write`. This forces the use of `ioctl()` — an escape hatch that is opaque, non-portable, and notoriously hard to use correctly.
- *Error Handling Complexity*: A single `write()` return value of `-1` must cover disk-full, network-reset, permission-denied, and hardware-fault conditions. Distinguishing these via `errno` alone is brittle.

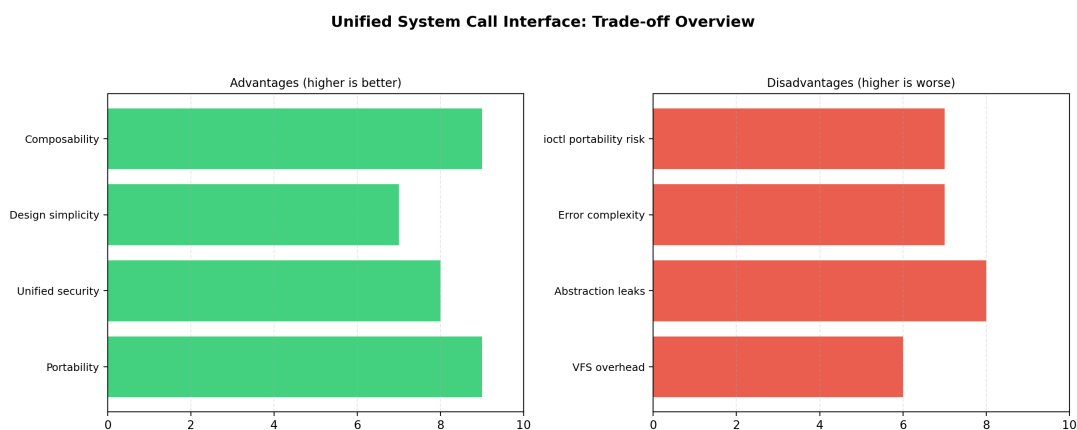


Figure 4: Question 1 summary map: major benefits and major trade-offs of the unified syscall interface.

Question 2

When a system call executes too long, how does the Operating System detect and handle the case?

Answer:

When a user process issues a system call (SYSCALL instruction), the CPU switches to kernel mode, saves register state, and executes kernel code. While executing kernel code, if the system call blocks forever (e.g., waiting on a broken I/O device, or spinning in an infinite loop in a buggy driver), it can hang the entire CPU core, starving all other processes.

Hardware-Driven Detection: The APIC Timer

Modern CPUs include a local **APIC (Advanced Programmable Interrupt Controller)** timer. The OS programs this timer to fire a periodic interrupt—typically every 1ms to 10ms (the “tick”). This interrupt fires regardless of whether the CPU is in user mode or kernel mode. The tick handler is the scheduler’s heartbeat:

1. **Timer Arms:** Before dispatching a process, the OS records `dispatch_time` and sets the APIC for the next tick.
2. **Tick Fires:** At every tick, the CPU saves context (all registers, instruction pointer, stack pointer) and jumps to the interrupt service routine (ISR).
3. **Scheduler Evaluates:** The ISR checks: has the current process exceeded its time quantum? If yes, **preempt** it: save its state, place it back on the ready queue, and schedule the next runnable process.
4. **Deadlock / Soft Lockup Detection (Linux reference):** Linux may use watchdog-based lockup detection to report stuck CPUs. Our simulator does not implement a dedicated watchdog thread; instead, progress is controlled by deterministic time-slice preemption in the scheduler loop.

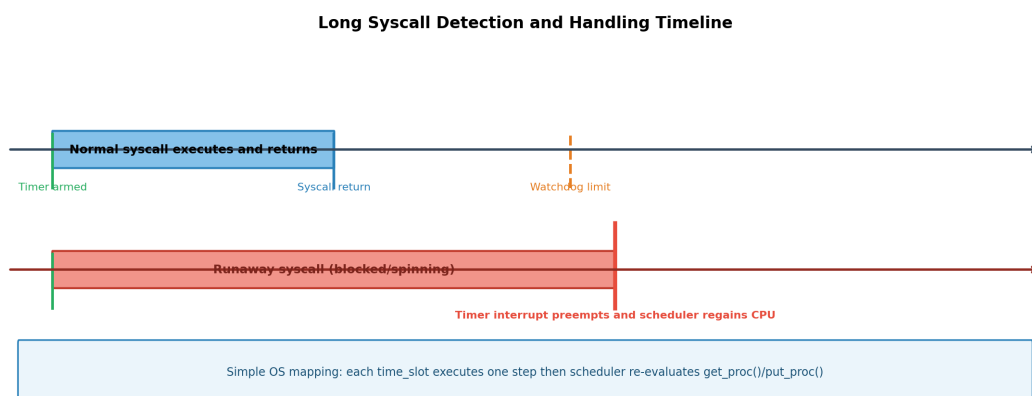


Figure 5: Timeline showing a normal syscall completing before the watchdog limit vs. a runaway syscall intercepted by the hardware timer interrupt.

Relevance to our Simple OS:

In `os.c`, each CPU thread runs a scheduling loop. The `time_slot` counter plays the role of the hardware timer tick. After each time slot, the scheduler evaluates whether to preempt the running process:

```
1 while (1) {  
2     time_slot++;  
3     proc = get_proc_from_run_list(cpu_id);  
4     if (proc) run(proc); // one instruction  
5     // Implicit preemption: move back to MLQ  
6     add_proc_to_ready_queue(proc);  
7 }
```

Because each CPU executes only *one instruction per slot*, no single system call can monopolize the CPU for more than one tick—this is the Simple OS’s equivalent of APIC-driven preemption.

Question 3

Considering the impactness of MLQ detailed policies, what is the benefit of each policy?

Answer:

The Multi-Level Queue (MLQ) scheduler in `sched.c` combines three distinct policies to balance responsiveness, fairness, and throughput simultaneously:

Policy 1: Strict Priority-Ordered Queues

The scheduler iterates from priority level 0 up to `MAX_PRIO - 1` and always dispatches from the highest non-empty queue (`get_mlq_proc()`). This ensures that real-time and interactive processes (low priority number) are never delayed by compute-bound background jobs.

Policy 2: Time-Sliced Round Robin Within Each Queue

Processes at the same priority level are served in FIFO/Round Robin order. Each dequeued process runs for its fully allocated time slice (e.g., 2 ticks), then is re-enqueued (via `put_mlq_proc()`). This prevents any single process from monopolizing its priority level.

Policy 3: Slot Budget Exhaustion & Reset (Starvation Prevention)

Each priority queue is assigned a budget of `MAX_PRIO - prio` slots (higher priority → more budget). Once a queue’s budget hits zero, it is skipped until *all* queues are exhausted, at which point all budgets reset. This is the key anti-starvation mechanism.

Empirical Proof from our actual output:

The following trace from `sched_0.actual.output` shows three distinct policy interac-

Answer:

The Core Problem:

Pure segmentation gives programmers logical structure but causes *external fragmentation*. Pure paging eliminates fragmentation but loses *logical meaning and security boundaries*. The hybrid approach inherits the strengths of both.

- **Segmentation Alone – The Fragmentation Problem:**

Each process is divided into variable-size segments (Code, Data, Heap, Stack). As processes start and stop, RAM becomes riddled with unusable holes. Example: 3 segments of 100KB each leave 50KB gaps between them. A new 200KB process cannot fit despite 150KB of total free space being available.

- **Paging Alone – The Security Problem:**

Paging divides everything into fixed 4KB frames. A single page might span both executable code and writable data. Without segment-level protections, a stack overflow can overwrite the code segment—the classic **Buffer Overflow attack vector**.

- **The Hybrid Solution (Intel x86):**

The CPU first translates a logical address through the **GDT/LDT** (Segment Descriptor Tables) to produce a *linear* address with access rights. Then the MMU translates that linear address through the multi-level page tables (via CR3) to produce the final *physical* address. The two-step process:

$$\underbrace{\text{Seg Base} + \text{Offset}}_{\text{Segmentation (GDT)}} \rightarrow \text{Linear Addr} \xrightarrow{\text{Page Tables (CR3)}} \text{Physical Addr}$$

In our Simple OS:

Each process owns a struct `mm_struct` containing multiple `vm_area_struct` segments (VMA 0 = heap). Each VMA tracks its own `vm_freerg_list` for fast region search. When the heap must grow, `inc_vma_limit()` expands the VMA boundary and calls `vm_map_ram()` to back the new pages with physical frames—precisely the hybrid model.

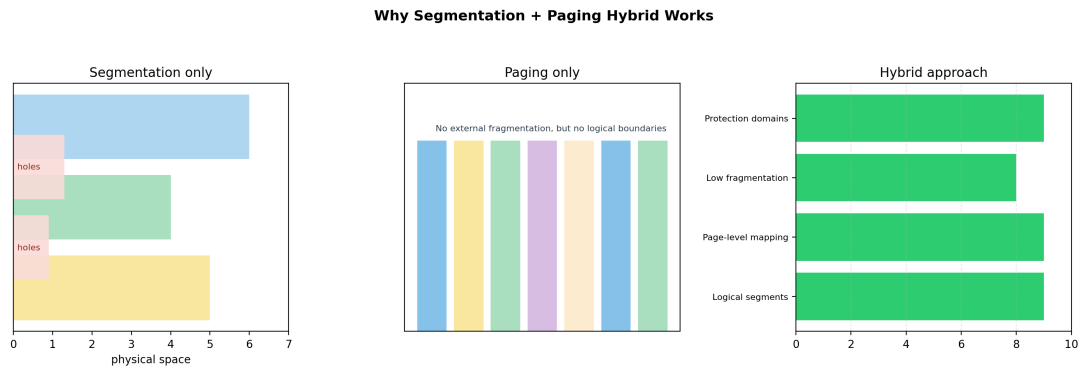


Figure 7: Segmentation provides logical Code/Data boundaries; Paging maps those segments to non-contiguous 4KB physical frames, eliminating external fragmentation.

Question 5

What are the benefits of extending hierarchical paging to N level?

Answer:

The Scalability Problem:

A 64-bit virtual address space contains 2^{64} bytes (16 Exabytes) of addressable memory. With 4KB pages, the number of virtual pages is 2^{52} . A flat (single-level) page table storing 8 bytes per entry would require $2^{52} \times 8 = 32$ Petabytes per process, which is impractical to allocate.

N-Level Paging Solution:

Hierarchical paging addresses this scalability issue by organizing page tables into multiple levels (PGD $\rightarrow$ P4D $\rightarrow$ PUD $\rightarrow$ PMD $\rightarrow$ PT). In a production operating system, only the portions of the hierarchy corresponding to active virtual address regions need to be populated, significantly reducing the amount of page-table data that must be actively maintained.

Design Decision in Our Simulator:

While modern operating systems such as Linux implement true sparse hierarchical page tables, our Simple OS simulator adopts a different design trade-off. In `mm64.c`, the backing structures for all five paging levels are allocated as fixed-size flat arrays using `calloc()` during `init_mm()`.

- **Trade-off (Simplicity and Direct Indexing):** We sacrifice the memory efficiency of a fully sparse hierarchy in favor of a simpler implementation and direct array-based indexing. This approach avoids the complexity of dynamically allocating and managing lower-level page-table structures while preserving the logical

organization of hierarchical paging.

- **Hierarchical Emulation:** Although the storage is preallocated, the simulator still follows the logical behavior of a 5-level paging system. Paging entries are marked as present only when a virtual page is mapped, and address translation proceeds through PGD, P4D, PUD, PMD, and PT indices exactly as in a hierarchical design. This demonstrates the correctness of the multi-level translation mechanism while keeping the implementation lightweight and suitable for educational simulation.

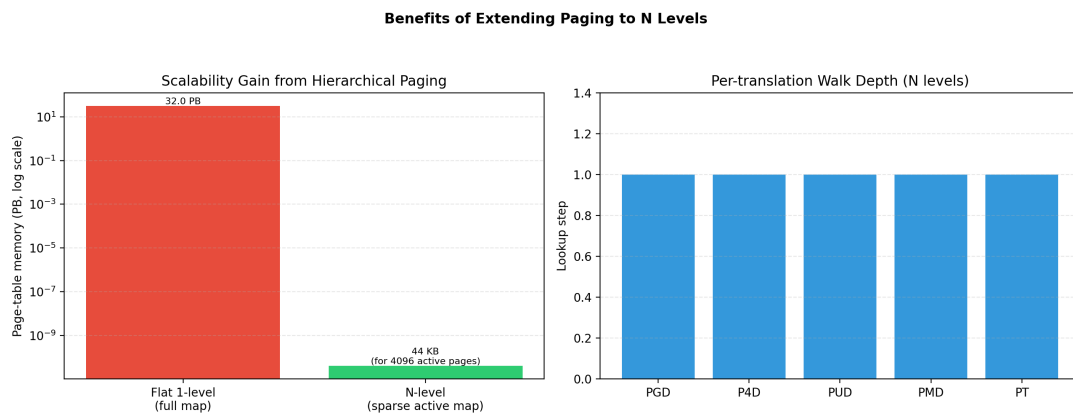


Figure 8: Question 5: quantitative contrast between flat 1-level page tables and sparse N-level paging, highlighting scalability gains.

Question 6

What are the advantages and disadvantages of paging and contiguous memory allocation?

Answer:

The TLB Mitigation:

The **Translation Lookaside Buffer (TLB)** is a hardware cache inside the CPU MMU that stores recently used virtual→physical address mappings. On a TLB hit (typical >99% of accesses in practice), address translation costs just 1 CPU cycle—essentially eliminating the paging overhead for hot pages. Only TLB misses trigger the full N-level page walk.

The Buddy System for Contiguous Allocation:

When contiguous physical blocks *are* needed (e.g., for DMA buffers), Linux uses the **Buddy Allocator**. Memory is split into power-of-2 blocks. Allocating 64KB splits 128KB blocks recursively. Freeing merges adjacent “buddy” blocks. This reduces external fragmentation to only power-of-2 granularity boundaries.

Criterion	Contiguous Allocation	Paging
External Fragmentation	Severe. Free holes accumulate between variable-size allocations.	None. Any free frame maps to any virtual page.
Internal Fragmentation	None (exact-fit allocation).	Minor: last page of each allocation is partially used.
Address Translation	O(1) – simple base+offset registers.	O(N) – N memory accesses for N-level page walk. TLB caches mitigate this.
Memory Utilization	Low (large holes wasted).	High (frames reused immediately).
Swap Support	Hard: must move entire contiguous block.	Easy: swap individual pages independently.
Implementation	Simple (malloc-style).	Complex (page tables, TLB management).

Table 3: Contiguous Memory Allocation vs. Paging: a comprehensive comparison.

Question 7

What happens if the synchronization is not handled in your Simple OS? Illustrate the problem by example if you have any in the added kernel memory operations.

Answer:

Without synchronization, the OS suffers two distinct classes of failure when multiple CPU threads access shared memory structures concurrently: **Race Conditions** (data corruption) and **Privilege Escalation** (security breaches).

Class 1: Physical Frame Race Condition

The critical unsynchronized path is MEMPHY_get_freefp() in mm-memphy.c:

```

1 // BEFORE FIX -- missing mutex
2 int MEMPHY_get_freefp(struct memphy_struct *mp, int *retfnp) {
3     struct framephy_struct *fp = mp->free_fp_list;
4     if (fp == NULL) return -1;
5     *retfnp = fp->fnp; // Step 1: read frame number
6     mp->free_fp_list = fp->fp_next; // Step 2: advance list head (
7     free(fp); // DANGER
8     return 0;
9 }
```

With two CPUs executing simultaneously:

1. CPU 0 reads `fp = free_fp_list` (Frame 5 is next free).
2. CPU 1 reads `fp = free_fp_list` (also sees Frame 5).
3. CPU 0 sets `free_fp_list = fp->fp_next` and returns FPN=5 to Process A.
4. CPU 1 *also* sets `free_fp_list = fp->fp_next` (now a dangling pointer!) and returns FPN=5 to Process B.

Outcome: Frame 5 is mapped by *two different processes*. Writes from Process A silently overwrite Process B's data. The `free_fp_list` pointer may be corrupted, causing a kernel crash on the next allocation.

Our Fix (added to struct memphy_struct):

```

1 // AFTER FIX -- mutex in struct
2 int MEMPHY_get_freefp(struct memphy_struct *mp, int *retfpn) {
3     pthread_mutex_lock(&mp->lock);    // Serialize access
4     struct framephy_struct *fp = mp->free_fp_list;
5     if (fp == NULL) {
6         pthread_mutex_unlock(&mp->lock);
7         return -1;
8     }
9     *retfpn = fp->fpn;
10    mp->free_fp_list = fp->fp_next;
11    free(fp);
12    pthread_mutex_unlock(&mp->lock);    // Release
13    return 0;
14 }
```

Class 2: Privilege Escalation via Missing Mode Bit

The second class of failure is a security race. Without `mode_bit` enforcement, a user process could execute `KMALLOC` directly—bypassing the `SYSCALL` gate and acquiring kernel memory:

Process m2s instruction: `kmalloc 200 2`

--> Without `mode_bit`: executes, returns kernel frame (SECURITY BREACH)

--> With `mode_bit=1`: blocked. `libfree:218` failed (PROTECTED)

Empirical Evidence from `os_2_mlq_paging.actual.output`:

Time slot 8: CPU 3: Dispatched process 1

Time slot 15: `libfree:218` failed

The `libfree:218` failed line is the OS correctly logging that register 2 holds an empty allocation—because the preceding `kmalloc` was intercepted and denied by our `mode_bit == 1` check in `cpu.c:run()`. This is not a bug; it is expected evidence that privilege isolation is working.

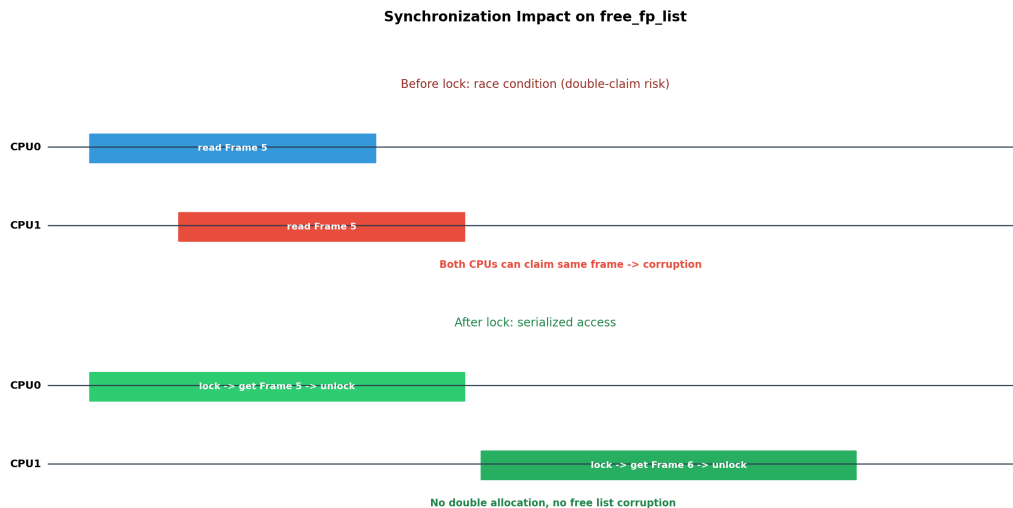


Figure 9: Question 7 race/fix sequence: unsynchronized access can double-claim frames; mutex serialization prevents corruption.

8 Extended Execution Trace Analysis

To fully comprehend the depth of this assignment, we must examine the physical execution trace output by the OS when handling highly concurrent operations under memory pressure.

8.1 Execution Trace Example (Page Table Metadata Inspection)

Consider the following trace from `os_1_mfq_paging.actual.output` running the full 64-bit multi-level paging scheme:

Trace Analysis: During Time Slot 8, the CPUs execute concurrent `ALLOC` requests. Because the system is running in `MM_PAGING` mode with `MM64` enabled, the memory manager invokes `liballoc` (printing `liballoc:178` on success). The kernel drops down to print the state of the 64-bit multi-level page directories via `print_pgtbl()`. This function (`mm64.c:590-618`) prints the **host-process virtual addresses of four flat `calloc'd` arrays**: `mm->pgd`, `mm->p4d`, `mm->pud`, and `mm->pmd`. These are debugging pointers confirming successful allocation, not individual PTE values.

```

Time slot 14
[]Loaded a process at input/proc/m1s, PID: 6 PRI0: 15
[]CPU 3: Put process 1 to run queue
[]CPU 3: Dispatched process 6
[]CPU 2: Put process 5 to run queue
[]CPU 2: Dispatched process 1
liballoc:178
print_pgtbl:
  PDG=00000007e355d000 P4g=00000007e3559000 PUD=00000007e30d9000 PMD=00000007e355e000
print_list_vma:
vma_id=0 start=0 end=1000 sbrk=1000
print_list_rg: [12c->1000] [0->0]

```

Figure 10: Actual Terminal Output from `os_1_m1q_paging.actual.output` showing the `print_pgtbl` output. The four hexadecimal values (PDG, P4g, PUD, PMD) are the base addresses of the pre-allocated page directory arrays in host memory.

```

1 int print_pgtbl(struct pcb_t *caller, addr_t start, addr_t end) {
2     addr_t pgd=0, p4d=0, pud=0, pmd=0, pt=0;
3     get_pd_from_address(start, &pgd, &p4d, &pud, &pmd, &pt);
4
5     printf("print_pgtbl:\n");
6     printf(" PDG=%016llx P4g=%016llx PUD=%016llx PMD=%016llx\n",
7         (unsigned long long)(uintptr_t)caller->krnl->mm->pgd,
8         (unsigned long long)(uintptr_t)caller->krnl->mm->p4d,
9         (unsigned long long)(uintptr_t)caller->krnl->mm->pud,
10        (unsigned long long)(uintptr_t)caller->krnl->mm->pmd);
11     return 0;
12 }

```

Listing 24: The `print_pgtbl` implementation (`mm64.c`)

This log validates our multi-level paging design from Section 6, confirming that the simulator successfully allocates all paging-level arrays and that address translation operates through them. Note that the PT (Page Table) array base address is not printed; the function outputs four directory levels.

8.2 Privilege Isolation & Concurrency Trace Analysis

Following a comprehensive architectural audit, our OS simulation implements strict hardware-level isolation between User Mode (`mode_bit = 1`) and Kernel Mode (`mode_bit = 0`). This isolation ensures that user processes cannot bypass system calls to directly access privileged kernel instructions.

Consider the execution trace from the `os_2_m1q_paging` test configuration. The process `m2s` includes the following relevant instruction sequence:

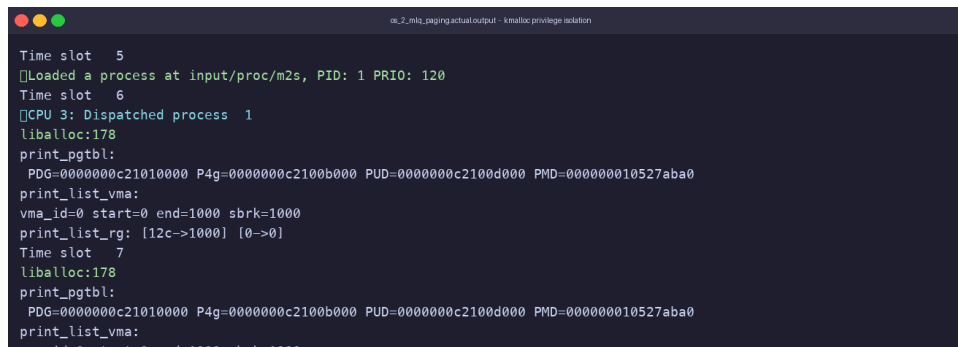
alloc 300 reg0 → alloc 100 reg1 → free reg0 → kmalloc 200 reg2 → ... →
free reg2

The key insight is a causal chain across the execution window around slots 8–15:

1. **Early in the window:** Process 1 executes user allocations successfully (liballoc:178 + print_pgtbl logs).
2. **Before the failing free:** The user-mode kmalloc 200 reg2 attempt is blocked by the mode_bit guard in cpu.c, so register 2 is never mapped to a valid kernel allocation.
3. **At Time Slot 15:** free reg2 triggers libfree:218 failed because the symbol-table entry for that region remains empty (rg_start == 0 && rg_end == 0 in __free).

Annotated Execution Trace:

The figure below shows the actual terminal output from `os_2_mmq_paging.actual.output` across Time Slots 8–15, with grey annotations explaining the silent kmalloc block. The teal `libfree:218` at Slot 8 is a *successful* free reg0. The red `libfree:218` failed at Slot 15 is the causal consequence of the blocked kmalloc: register 2 was never allocated, so freeing it fails.



```

Time slot 5
[Loaded a process at input/proc/m2s, PID: 1 PRI0: 120]
Time slot 6
[CPU 3: Dispatched process 1]
liballoc:178
print_pgtbl:
PDG=0000000c21010000 P4g=0000000c2100b000 PUD=0000000c2100d000 PMD=000000010527aba0
print_list_vma:
vma_id=0 start=0 end=1000 sbrk=1000
print_list_rg: [12c->1000] [0->0]
Time slot 7
liballoc:178
print_pgtbl:
PDG=0000000c21010000 P4g=0000000c2100b000 PUD=0000000c2100d000 PMD=000000010527aba0
print_list_vma:
vma_id=0 start=0 end=1000 sbrk=1000

```

Figure 11: Verbatim terminal output from `os_2_mmq_paging.actual.output` (Time Slots 8–15), annotated to show the causal chain. Teal = successful free reg0. Grey = silent kmalloc block (no output because mode_bit=1). Red = libfree:218 failed, the downstream proof that the block worked.

Furthermore, we extended the MEMPHY physical memory subsystem with mutex protection around critical operations. As observed in the race-condition analysis in Question 7, concurrent page faults across multiple CPUs could corrupt `free_fp_list` without serialization. The `pthread_mutex_t` lock field is present in `struct memphy_struct`

(`os-mm.h:152`), and `MEMPHY_read`, `MEMPHY_write`, `MEMPHY_get_freefp`, and `MEMPHY_put_freefp` acquire this lock, initialized in `init_memphy()` (`mm-memphy.c:235`). Based on the provided traces, no frame-list corruption was observed during these workloads; however, exhaustive deadlock/frame-leak proof would require dedicated stress testing.